

Verifier-Guided Prompting for Intent-to-Configuration Translation: A Preregistered NetOps Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory trial to test whether constrained prompting plus a deterministic verifier-guided generate→verify→repair loop (one allowed repair) increases deterministic-verifier semantic-correctness when translating operator natural-language intents into low-level device configurations. Design: N=150 synthetic intents sampled from a deterministic generator, shared_initial_candidate generation semantics, model = gpt-5-mini, deterministic validator = deterministic_netops_validator_v5, one initial generation per intent shared across baseline and guarded conditions, guarded pipeline permitted ≤1 repair call. Results (artifact-grounded): baseline = 149/150 (99.333%); guarded = 150/150 (100.0%); paired absolute difference = 0.006667 (≈0.67 percentage points); exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.00, 0.02] (10,000 resamples, seed = 1729). Provider accounting: completed episodes = 300; model_calls_used = 151; repair-stage invocations = 1. Interpretation: on this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow produced a numerically negligible and statistically non-significant improvement over a near-ceiling baseline. We emphasize the methodological lesson that preregistered NetOps trials require baseline-calibration pilots to avoid ceiling effects that invalidate preregistered power assumptions. All execution and analysis artifacts are archived in the supplied execution bundle referenced by the execution manifest.

I. INTRODUCTION

Natural-language intent interfaces aim to reduce operator burden by letting humans express desired network changes as free text that is automatically translated into executable device configurations. Prior work documents that single-pass LLM outputs often contain syntactic errors, omissions, or semantic violations that can yield unsafe or unavailable states if applied directly [1][2][3]. A commonly proposed defense is a generate→verify→repair architecture in which an LLM produces a candidate, a deterministic verifier checks intent-level invariants or formalized constraints, and an automated repair (or human gate) corrects violations before execution [4][5]. Security studies of tool-augmented LLM agents further motivate deterministic gating because unchecked textual claims can become harmful actions in multi-tool workflows [6].

Research question. After an autonomous literature review and preregistration we posed a confined confirmatory question: does constrained prompting (structured templates with explicit semantic slots) combined with deterministic verifier-guided iterative feedback materially increase verifier-level semantic correctness of NL→configuration translation versus

an unconstrained baseline under a paired design? The trial (study_cand_2026_b2_v1) was preregistered and frozen before any model outputs were observed.

Contributions. (1) A fully preregistered, artifactized paired confirmatory trial measuring deterministic-verifier pass/fail on a programmatically generated synthetic corpus (N=150 paired intents). (2) Artifact-backed outcomes showing that, under shared_initial_candidate semantics and a one-repair budget, the guarded pipeline raised verifier pass rate from 149/150 to 150/150 (+0.67 pp), a numerically negligible and statistically non-significant improvement (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). (3) A methodological recommendation that preregistered NetOps trials include baseline-calibration pilots and explicitly specify generation semantics (shared_initial_candidate vs independent sampling) because semantics materially change the estimand and interpretation.

II. RELATED WORK

Related literature falls into three strands. First, intent-to-configuration work demonstrates that converting free-text intents into verifier-sound artifacts is difficult and reports modest semantic-correctness on benchmarks, motivating post-generation checks [1][2]. Second, generate→verify→repair architectures—ranging from verifier-in-the-loop fine-tuning to policy-guided verifier feedback—have shown correctness improvements for infrastructure-as-code and configuration repair, but reported gains vary with baseline model competence, repair budget, and sampling semantics [4][7]. Third, security and systems analyses have demonstrated claim-to-action vulnerabilities where multi-tool LLM pipelines can convert false textual claims into concrete harmful actions, arguing for auditable gating and verifier-aware planning [6][8].

How this study situates. Our trial operationalizes a narrowly specified guarded architecture and emphasizes preregistration, paired inference, and deterministic-verifier endpoints. Key contrasts with prior reports that observed larger guarded-vs-baseline gains are: (a) shared_initial_candidate execution semantics (one initial sample shared across conditions), (b) a strict one-repair budget per intent, (c) a single tested model (gpt-5-mini), and (d) a synthetic verifier-capable task bank restricted to small topologies. These design choices reduce the maximal observable marginal benefit of the guarded repair under our estimand and explain why previously reported larger gains can coexist with our bounded result [4][7].

III. METHODOLOGY

Overview and preregistration. We preregistered `study_cand_2026_b2_v1` and froze an analysis plan prior to any model calls. The design is paired-binary: each sampled intent yields one shared initial candidate used to evaluate both the baseline and guarded conditions (`shared_initial_candidate` semantics). The primary estimand was the `paired_success_rate_difference_guarded_minus_baseline` and the confirmatory test was an exact two-sided McNemar on the paired contingency table. The preregistration specified $N=150$ confirmatory intents, inclusion/exclusion rules, a one-repair budget for guarded episodes, a deterministic retry policy for provider timeouts (one retry), and an analysis plan that included a 10,000-resample paired bootstrap (`bootstrap_seed = 1729`) to produce percentile CIs. These preregistered elements are archived; the registered preregistration SHA-256 is recorded in the execution manifest.

Task generation, intent families, and prechecks. Confirmatory tasks were produced deterministically by `deterministic_netops_task_generator_v5`. The generator emits intent payloads with: (i) an explicit natural-language intent string, (ii) an `initial_state` and `expected_final_state`, (iii) an ordered `required_sequence` of field-level operations (e.g., `shutdown`→`mtu`→`restore`), (iv) policy constraints (`allowed_touched_fields`, `protected_interfaces`, `final_group_constraints`), and (v) a declared difficulty label. The confirmatory corpus was balanced across four intent families: safe sequence reachability/isolation, ACL-like access changes, VLAN/MTU sequence manipulations, and uplink switchover patterns. Topologies were restricted to verifier-capable small networks (4–32 nodes) to ensure deterministic validator coverage. Prior to model calls we ran deterministic unit-tests on the mapping/transformation code and a rule-based ambiguity detector; items flagged as ambiguous or failing parser/unit-tests were excluded per the preregistered exclusion rules. In this execution none of the confirmatory items were excluded.

Execution adapter, sampling semantics, and explicit baseline interpretation. The `hosted_netops_gvr_v1` adapter enforced the `shared_initial_candidate` semantics required by the preregistration: for each intent the adapter performed exactly one initial model generation (the shared candidate) and used that identical candidate when scoring both the baseline and guarded conditions. The guarded pipeline could issue at most one additional repair call only if the deterministic validator failed the shared initial candidate. Because of this architecture, the baseline success rate measures the validity of the single initial sampled candidate; the guarded vs baseline contrast therefore quantifies the marginal effect of a single allowed repair applied to the identical sampled candidate rather than the effect of independently re-sampling first-pass candidates per condition. We include this precise sampling-to-estimand mapping in Methods to eliminate interpretive ambiguity about whether different first-pass generations were drawn for each condition.

Model configuration and nondeterminism. Declared

model: `gpt-5-mini` (provider record: `openai_responses`). The archived `execution/model_configuration.json` records `declared_model_version = "gpt-5-mini"` and explicitly records `temperature = null` (unset). Importantly, the adapter and preregistration did not enforce a numeric temperature value or a fixed random seed; therefore model outputs may be nondeterministic across independent API sessions. We state this here in Methods and repeat it in the Disclosure Statement to comply with preregistration/runtime-parameter reporting requirements.

Repair budget, session isolation, and provider auditing. The guarded repair policy allowed at most one repair invocation per intent and the adapter maintained strict session isolation: each model call ran in a fresh API session (no cross-call state reuse). Provider auditing tracked `hosted_model_call_event_count` and cache statistics; the deterministic reconciliation artifact reports `completed_hosted_model_call_event_count = 151`, `cache_miss_count = 151`, and `stage_counts = {shared_initial_generation: 150, repair: 1}`. The adapter implementation enforces the preregistered retry rule (one retry after transient provider failure). All provider-call metadata and per-call runtime parameters were archived for audit.

Deterministic validator, scoring rule, and diagnostic trace structure. We used `deterministic_netops_validator_v5` (`validator_version = 5.0`) as a deterministic oracle for the primary binary endpoint. The validator implements a `full_intent_constraint_validation` rule that: (a) parses the LLM-produced candidate into normalized command sequences, (b) computes `parsed_command_count` and compares against `required_command_count` derived from the task’s `required_sequence`, (c) enforces workflow and group constraints (e.g., `make-before-break`, `must_be_down_for_fields`), and (d) returns a boolean valid flag plus structured diagnostics (`violation_codes`, `observed_touched_field_count`, `normalized_configuration`, `final_state` snapshot). Scoring for the confirmatory endpoint used the validator’s boolean pass/fail; the richer diagnostics were used for failure-mode analysis and are archived as validator traces for each episode.

Inclusion/exclusion, contamination controls, and pre-execution unit-tests. The preregistered inclusion rules required that an intent pass the deterministic ambiguity detector and that mapping code successfully produce verifier inputs. We executed deterministic unit-tests for mapping code and verifier-input transformations before batch evaluation; these tests passed. Contamination checks (session-leak detection, duplicate-response entropy checks) were run; `contamination_summary.csv` recorded zero flags. Deterministic reconciliation confirms episode accounting consistency and reports no provider-call failures.

Statistical analysis posture (confirmatory vs exploratory). The methodology adheres to the preregistered multiplicity and missingness rules: the primary confirmatory contrast is guarded vs baseline at $\alpha=0.05$ (exact McNemar two-sided) using `complete_pair_primary` (exclude incomplete pairs). Secondary contrasts were preregistered (constrained vs baseline,

guarded vs constrained) with Bonferroni correction; in practice shared_initial_candidate semantics and adapter limits constrained the realized secondary contrasts. Analysis artifacts include paired_contingency_table.csv, condition_summary.csv, and a paired bootstrap used to compute the 95% CI (bootstrap_resamples = 10000, bootstrap_seed = 1729). All analysis code, seeds, and artifacts are archived for reproducibility.

Reproducibility artifacts and archival pointers. The full execution manifest, raw model-call logs, per-call runtime metadata, task-generator seeds, verifier inputs/verdicts, and analysis outputs are archived in the execution bundle referenced by the execution manifest. Deterministic reconciliation and provider accounting artifacts record model_calls_used = 151 and stage-level counts; these artifacts support replication of the paired accounting and the precise sampling semantics used in this trial.

IV. RESULTS

Execution and provider audit. The run completed exactly as planned: completed_episode_count = 300 and complete_pair_count = 150. Provider-call accounting (from the archived execution manifest) shows model_calls_used = 151, completed_hosted_model_call_event_count = 151, failed_hosted_model_call_event_count = 0, cache_miss_count = 151, and stage_counts = {shared_initial_generation: 150, repair: 1}. Contamination checks raised no flags and all unit-tests that gate mapping and verifier-input transformations passed prior to batch execution.

Primary contingency and per-condition rates. The paired contingency table (guarded $\times$ baseline) observed in the archived analysis artifacts was: $n_{11} = 149$ (both-pass), $n_{10} = 1$ (guarded-only-pass), $n_{01} = 0$ (baseline-only-pass), and $n_{00} = 0$ (both-fail). These counts yield per-condition success totals baseline_success_count = 149/150 = 0.9933333333333333 and guarded_success_count = 150/150 = 1.0, and an absolute paired difference (guarded - baseline) = 0.00666666666666671 (≈ 0.67 percentage points). The preregistered exact two-sided McNemar test on these paired counts produced $p = 1.0$. The paired bootstrap-percentile 95% confidence interval computed with 10,000 resamples (bootstrap_seed = 1729) is [0.00, 0.02]. All these numeric values are taken verbatim from the archived analysis files (analysis/paired_contingency_table.csv and analysis/condition_summary.csv).

Repair-stage diagnostics and revision iterations. The guarded pipeline issued a repair-stage model call in exactly one episode (1/150 tasks), matching the provider audit (stage_counts.repair = 1). The archived scoring traces indicate that this single repair invocation converted an initially verifier-failing candidate into a verifier-accepted normalized_configuration; the validator logs include before/after normalized text, violation_codes, and parsed_command_count fields for that episode. Across representative archived traces (tasks 000001–000006) the parser- and validator-level diagnostics show parsed_command_count values consistent with the required_command_count reported by the generator (examples

in the archive show parsed_command_count values of 2, 4, and 6 for representative easy-to-hard cases). Consistent with the rarity of repairs, the median revision iterations per guarded episode = 0 (IQR = 0), and only the single discordant pair required a guarded repair under the preregistered one-repair budget.

Representative artifact-grounded examples. Representative archived task traces illustrate the variety of structural difficulty encoded by the task generator. For example, task-000001 (generator_seed = 1) required a four-step shutdown $\rightarrow$ change $\rightarrow$ restore sequence; the shared-initial candidate and both condition responses matched the required four-command sequence and the validator reported parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, and valid = true. Other representative traces (e.g., a two-command make-before-break uplink switchover and a six-command paired-MTU change) demonstrate that the corpus includes both relatively compact and more complex required sequences; these representative artifacts (responses and validator traces) are archived and cited in the execution manifest.

Sensitivity and missingness outcomes. The preregistered sensitivity analysis planned to treat missing guarded outputs as failures was not exercised because failed_hosted_model_call_event_count = 0 and no unscorable episodes occurred. The missingness summary in the archive reports complete_pairs = 150 and all related counts = 0 for failed or unscorable episodes.

Compact evidence tables and artifacts. Table 1 (paired contingency) and Table 2 (execution audit summary) in the manuscript reproduce the exact archived CSVs (analysis/paired_contingency_table.csv and analysis/condition_summary.csv). The analysis artifacts also record bootstrap parameters (bootstrap_resamples = 10000; bootstrap_seed = 1729) and the paired-analysis log entry that marks the completion of the confirmatory analysis.

Interpretation grounded in archived artifacts. On this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow yielded a numerically small absolute improvement (+0.67 pp) over an already near-ceiling baseline. The exact McNemar $p = 1.0$ and the bootstrap CI that includes zero indicate no statistically supported confirmatory benefit under the preregistered estimand. The empirical facts driving this conclusion are: (a) an observed baseline success rate $\approx 99.33\%$ that left essentially no headroom for a large absolute increase, and (b) a single, rare repair invocation that produced the only discordant pair. These artifact-grounded observations motivate the methodological recommendation to perform baseline-calibration pilots prior to confirmatory sampling to avoid underpowered trials caused by ceiling effects. All numeric statements above and the diagnostic traces referenced are archived in the execution bundle and analysis artifacts cited in the execution manifest.

V. DISCUSSION

We expand the discussion with concrete, artifact-grounded diagnostics, methodological implications, and operational interpretations that directly follow from the archived execution and analysis artifacts.

Ceiling effect, discordance structure, and inferential consequence. The contingency structure ($n_{11}=149$, $n_{10}=1$, $n_{01}=0$) makes explicit why the exact McNemar test returns $p=1.0$: almost all paired episodes agree and only a single discordant pair favors the guarded condition. Under McNemar’s paired-proportion logic the test’s power derives from the number of discordant pairs; with only one discordant pair there is effectively no evidence for a systematic directional effect. The paired bootstrap CI $[(0.00, 0.02]$, $\text{bootstrap_seed}=1729$, $\text{resamples}=10000$) quantifies the same intuition by showing that plausible values for the absolute improvement under resampling include zero and are bounded above by ≈ 2 percentage points. Both diagnostics are reported verbatim from `analysis/paired_contingency_table.csv` and `analysis/condition_summary.csv` and therefore trace directly to archived artifacts rather than post hoc interpretation.

Mechanistic explanation via execution semantics. The `shared_initial_candidate` semantics materially reduce the pathway by which the guarded pipeline can improve outcomes: because the identical initial candidate is used to evaluate both baseline and guarded conditions, any potential advantage of the guarded pipeline depends entirely on the single allowed repair call converting that specific initial candidate into a verifier-pass. This is reflected in provider accounting: `stage_counts` show `shared_initial_generation=150` and `repair=1`, and `model_calls_used=151`. Practically, when baseline initial candidates are already highly likely to pass the verifier (here $\approx 99.33\%$), the repair budget is rarely exercised and the guarded condition has almost no opportunity to improve aggregate pass rates. This mechanism explains the observed near-ceiling result and illustrates how sampling semantics and repair budgets interact with baseline competence to determine observable effect sizes.

Design consequences for preregistration and power calibration. The preregistration targeted detection of a 15 percentage-point absolute improvement with $N=150$. That calculation assumed substantially lower baseline competence and therefore a larger expected discordant mass. Our artifact-grounded outcome demonstrates the risk of committing to a confirmatory N without an empirical baseline check: baseline calibration reduces the chance of an underpowered or effectively vacuous confirmatory test caused by ceiling effects. Concretely, a practical pilot procedure (consistent with our recommendations and implementable from the archived deterministic task generator) is: sample 20–40 tasks from the planned generator with the intended prompt styles; measure baseline verifier-pass rate on those pilot tasks; if baseline $>\sim 85\text{--}90\%$ either increase task difficulty or enlarge N and/or adopt independent-sampling semantics or a larger repair budget. These procedural steps require no new experimental artifacts and can be implemented

with the provided generator seeds and unit-tested mapping code archived in the execution bundle.

Operational affordances beyond the binary endpoint. Even when aggregate verifier-pass improvements are negligible, the guarded pipeline provides operationally valuable artifacts that the binary pass/fail metric does not capture. The deterministic validator returns structured diagnostics (`normalized_configuration` strings, `parsed_command_count`, `required_command_count`, `violation_codes`) for each episode; these are visible in validator traces for representative tasks (e.g., `task-000001..000006`). In practice these strings enable: (1) auditable diffs for operator review, (2) concise failure taxonomies to prioritize human triage, and (3) deterministic gating rules implementable as fail-stop or auto-remediation policies. The run-level audit (`completed_hosted_model_call_event_count=151`; `failed_hosted_model_call_event_count=0`) proves these artifacts were produced consistently across the sample and are therefore operationally available in the archived bundle for downstream tooling.

Trade-offs among sampling semantics, repair budgets, and estimands. Our evidence clarifies three non-identical estimands that practitioners must choose between and that should be explicit in preregistrations: (A) `shared_initial_candidate` marginal-repair effect (our estimand), (B) independent-sampling expected-performance difference (draw independent initial samples per condition), and (C) repair-robustness with expanded repair budgets (allow multiple iterative repairs). Estimand A is conservative with respect to repair effectiveness because it fixes the initial draw; estimand B typically yields larger observable differences when model output variance is high; estimand C quantifies diminishing returns from additional repair attempts. The archived artifacts make it possible to re-run variants B or C (subject to provider budget and adapter constraints) starting from the same deterministic task generator seeds and the stored `model_configuration` metadata; this preserves reproducibility while acknowledging that different, equally defensible operational choices lead to different scientific answers.

Reproducibility notes tied to concrete artifact fields. Key numeric analysis parameters are archived and should be used by any reproducer: `bootstrap_resamples=10000` and `bootstrap_seed=1729` (`analysis/analysis_log.jsonl`), `model_calls_used=151` and `stage_counts` (`analysis/deterministic_reconciliation.json`), and `declared_model_version="gpt-5-mini"` with `model_configuration.json` recording `temperature=null` (unset). Because temperature was unset and no fixed random seed was enforced, repeated independent provider sessions can produce nondeterministic outputs; reproductions seeking exact token-level matches must therefore control sampling parameters or accept equivalent statistical replications rather than byte-identical replays. All of these facts are documented in the execution manifest and `model_configuration.json` and are cited here to avoid ambiguity about what archive-grounded reproduction entails.

Threats to validity revisited with artifact grounding. Internal validity: the adapter’s session-isolation and provider-call audit (no cross-condition cache hits observed) reduce concerns about prompt-history leakage; these checks are recorded in `deterministic_reconciliation.json`. Construct validity: deterministic validator pass/fail reflects the predeclared invariants from the generator’s `task_payload`; mapping-code unit-tests passed pre-execution but any gaps in verifier coverage remain a construct threat—this is why we limited claims to the verifier-capable invariants and reported representative validator violation fields when present. External validity: the single-model constraint (gpt-5-mini) and small-topology synthetic corpus restrict generalization; these are enumerated in the preregistration and in the limitations section and are supported by the `model_configuration` and `task-manifest` artifacts. Conclusion validity: the mismatch between preregistered power assumptions and the observed baseline is a concrete illustration of how inferential conclusions depend on realized effect sizes and discordant-pair counts, both archived in the `paired_contingency_table.csv`.

Practical recommendations for practitioners and experimenters. (1) Always run a short baseline-calibration pilot using the final prompt templates and generator to estimate baseline competence and select N, difficulty, and repair budgets accordingly. (2) Make generation semantics explicit in preregistration: `shared_initial_candidate` vs independent sampling lead to different operational decisions and must match intended deployment semantics. (3) When verifier coverage is partial, report both binary pass/fail and the structured validator diagnostics to allow operators to triage and to enable metric-based comparisons across verifier designs. (4) Consider multi-arm designs that vary repair budgets and sampling semantics to estimate diminishing returns and to align statistical power with expected discordance rates. The provided execution bundle contains deterministic generator seeds, unit-tested mapping code, and validator traces sufficient to implement each recommendation without inventing new artifacts.

Summary. The expanded discussion ties our artifact-grounded numerical outcomes to concrete methodological mechanisms (sampling semantics and repair budget), diagnostic artifacts (validator traces and audit counts), reproducibility parameters (bootstrap seed, model metadata), and explicit experimental recommendations. These points are strictly supported by the archived execution and analysis artifacts and do not alter empirical claims reported elsewhere in the manuscript.

VI. LIMITATIONS

- Single-model constraint: experiments used only gpt-5-mini; results do not generalize across models or sizes.
- Synthetic corpus and scale: tasks were programmatically generated and limited to verifier-capable toy-to-small topologies (4–32 nodes); external validity to production WANs and heterogeneous vendor CLIs is untested.
- `Shared_initial_candidate` semantics and execution contract limited repair budget to at most one guarded repair

call per intent; different generation semantics or multiple-repair budgets may yield different outcomes.

- Ceiling effect and preregistration mismatch: baseline correctness was far higher than the pilot assumptions used for power calculations (planned detectable effect = 15 percentage points), reducing the trial’s ability to demonstrate benefit and illustrating sensitivity to baseline calibration.
- Verifier coverage and specification translation: the study measures deterministic validator pass/fail on the preregistered invariants but does not claim safety guarantees beyond the deterministic validator’s modeled properties.
- Security and adversarial robustness: the experiment did not evaluate adversarial prompt-injection or adversary-driven intents; separate targeted studies are required to assess claim-to-action vulnerabilities in agentic NetOps workflows.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory trial comparing baseline free-text prompting with constrained-template prompting plus a single verifier-guided repair under `shared_initial_candidate` semantics. On a deterministic synthetic corpus with gpt-5-mini and deterministic `netops_validator_v5`, the guarded pipeline increased verifier pass rate from 149/150 to 150/150 (≈ 0.67 pp), a numerically tiny and statistically non-significant difference (exact McNemar $p = 1.0$; 95% bootstrap CI = [0.00, 0.02]). The principal scientific lesson is methodological: preregistered NetOps trials should include baseline calibration to avoid ceiling effects and preserve statistical power. Technically, our artifacts bound the marginal benefit of a one-repair guarded pipeline under `shared_initial_candidate` semantics; evaluating independent-sampling semantics, larger repair budgets, model diversity, and production-scale topologies remains important future work.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock under the archived master prompt (`provenance/master_prompt.txt`, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Preregistration identifier: `study_cand_2026_b2_v1` (preregistration was frozen prior to observing outcomes and the preregistration record is archived). Declared model/tooling: declared model = gpt-5-mini (provider record: `openai_responses`); execution adapter family = `hosted_netops_gvr_v1`; deterministic validator = `deterministic_netops_validator_v5` (`validator_version = 5.0`). Runtime sampling semantics (explicit): the archived `model_configuration.json` records `temperature = null` (unset); because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions. Autonomy and human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy

lock; after the lock the autonomous system performed literature review, study selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No human manually edited technical content, analyses, figures, captions, or references after the autonomy lock. Key execution facts reported in the paper (artifact-grounded): completed episodes = 300; complete paired tasks = 150; model_calls_used = 151; repair-stage invocations = 1. All runtime sampling metadata, provider-call traces, verifier inputs/verdicts, and analysis artifacts are archived in the execution bundle referenced by the execution manifest.

REFERENCES

- [1] <https://doi.org/10.1002/nem.2313>
- [2] <https://doi.org/10.1145/3786583.3786898>
- [3] <https://arxiv.org/abs/2604.22513>
- [4] <https://doi.org/10.1145/3605764.3623985>
- [5] <https://doi.org/10.1145/3656454>
- [6] <https://doi.org/10.1002/widm.70111>